

FLAT ORIGAMI IS TURING COMPLETE

THOMAS C. HULL AND INNA ZUKHREVICH

ABSTRACT. *Flat origami* refers to the folding of flat, zero-curvature paper such that the finished object lies in a plane. Mathematically, flat origami consists of a continuous, piecewise isometric map $f : P \subseteq \mathbb{R}^2 \rightarrow \mathbb{R}^2$ along with a layer ordering $\lambda : P \times P \rightarrow \{-1, 1\}$ that tracks which points of P are above/below others when folded. The set of crease lines that a flat origami makes (i.e., the set on which the mapping f is non-differentiable) is called its *crease pattern*. Flat origami mappings and their layer orderings can possess surprisingly intricate structure. For instance, determining whether or not a given straight-line planar graph drawn on P is the crease pattern for some flat origami has been shown to be an NP-complete problem, and this result from 1996 led to numerous explorations in computational aspects of flat origami. In this paper we prove that flat origami, when viewed as a computational device, is Turing complete, or more specifically P-complete. We do this by showing that flat origami crease patterns with *optional creases* (creases that might be folded or remain unfolded depending on constraints imposed by other creases or inputs) can be constructed to simulate Rule 110, a one-dimensional cellular automaton that was proven to be Turing complete by Matthew Cook in 2004.

CONTENTS

1. Introduction	1
2. Conventions and preliminaries	2
3. Logic gates and other gadgets	6
4. Folding Rule 110	12
5. Conclusion	14
References	17

1. INTRODUCTION

Origami, the art of paper folding, has lately been a source of inspiration for applications in mechanical engineering [FS21, MHM⁺17], materials science [SEM⁺14, LSE⁺18], and architecture [MSP⁺18]. Helping this interest has been the rise of *computational origami*, which studies computational questions that emerge from the folding of paper, as a field in computational and combinatorial geometry [DO07]. Of particular interest has been *flat origami*, where a two-dimensional sheet of paper, or all of $\mathbb{R}^2$, is folded into a flat object, back into the plane without stretching, tearing, or self-intersecting the paper. For example, in 1996 Bern and Hayes proved that the decidability question of whether a given crease pattern can fold flat is NP-hard [BH96]. However, because of the difficulty in rigorously modeling flat origami, a hole in their proof remained undetected for 20 years until Kitayama et al. repaired and strengthened their proof in 2016 [CD⁺16].

The fact that flat origami is so computationally difficult may seem strange and contradictory, but this is because it involves a complicated interaction between local and global phenomena. For example, Kawasaki's Theorem (see Theorem 3 below) states that a set of creases which all meet at a vertex can fold flat if and only if the alternating sum of its sector angles is zero. However, this condition is not sufficient for a multiple-vertex crease pattern or even crease patterns with non-intersecting lines to fold flat; for this, it is necessary to consider global properties. For instance, if we make two parallel creases on a sheet of paper, they can fold flat in the same direction only if the distance between the creases is large enough to fit the two flaps. Thus even though each individual crease can flat-fold, the two creases can interfere with one another. With more complicated crease patterns these interactions become more and more complicated, with a complete

analysis requiring an understanding of the order of different layers of the paper (see Section 2). Thus the global structure of the crease pattern massively increases the complexity of flat origami.

In a similar vein, we can consider finite cellular automata: “machines” with cells, each of which can have only finitely many states, and where cells shift states based on the states of their neighbors. If a finite cellular automaton only has finitely many cells then it’s a finite state machine, with very little computational power. But cellular automata with infinitely many cells, even very restricted ones where each cell only sees two neighbors, are extremely computationally powerful. Given that flat-folding origami is much more complex than the cellular case, even for only finitely many folds, the question arises: How complex is flat-folding origami when it is placed in an infinite sheet, with infinitely many creases allowed?

A problem containing an infinite amount of information can be arbitrarily complex. We therefore decide to limit the problem. We start with an *origami tessellation* (i.e. a periodic crease pattern that is possible to fold flat) on an infinite sheet of paper. Then we allow ourselves to modify only finitely many of the creases in the tessellation and seek to determine the complexity of the problem: Can this new crease configuration fold flat? The idea is that our tessellation would model the behavior of a cell in a one-dimensional cellular automaton, with the cells going across the paper, left-to-right, and the “result of the computation” going down the paper. If the computation terminates, the “modification” would be constrained to a finite portion of the paper; if it did not then the “necessary modification” for the pattern to flat-fold would go off to infinity.

In this paper we consider a very simple example of a cellular automaton called Rule 110 (see Section 2.2 for a more detailed definition). This cellular automaton consists of an infinite line of cells, each of which can be “on” or “off.” At each click of the computation, a cell changes its state based on its current state and the states of its two neighbors. Thus the “computational history” of the entire process can be visualized as a two-dimensional grid of squares, with each row being a state of the computation and each column recording the sequence of states that a cell went through. In [Coo04], Cook showed that Rule 110 is Turing-complete, and not long after Neary and Woods [?, Theorem 1] showed that Rule 110 is logspace-complete for $\mathbf{P}$. In this paper we prove the following theorem:

Theorem 1. *It is possible to construct an origami tessellating cell which can model Rule 110: i.e. the output of the cell is determined from the three inputs via Rule 110.*

In particular, we can take the entire history of a computation using Rule 110 and model it using a tessellating cell in origami. See Section 4.2 for a more in-depth discussion. As a corollary, we get the following statement which is more specific than the claim made by this paper’s title of origami being Turing complete:

Corollary 2. *Tessellating flat-foldable origami is logspace-complete for $\mathbf{P}$.*

Our approach is to make use of *optional creases* in our crease patterns to help encode Boolean variables and design logic gates, an approach that has been used in prior work to explore the complexity of origami [CD⁺20]. In Section 2 we will formally define our model of flat-foldable origami, define Rule 110, and establish conventions in our approach. In Section 3 we will define and prove the correctness of the origami gadgets we will use to transmit Boolean signals and simulate logic gates. In Section 4 we will put our gadgets together to simulate cellular automata, in particular Rule 110.

Acknowledgements. The first author is supported in part by NSF DMS-1906202 and DMS-2347000. The second author is supported in part by NSF DMS-1846767. The authors would also like to thank Damien Woods for helpful comments and the reference to his paper [?].

2. CONVENTIONS AND PRELIMINARIES

What it means to “fold a piece of a paper” or for a crease pattern to “fold flat” is fairly intuitively obvious. However, as with many intuitively obvious things, it turns out to be fairly difficult to write rigorously. This section lays out the rigorous definitions and background necessary for the technical analysis in the paper, but it is not necessary for a first (or even a third) understanding, and many readers can freely skip it.

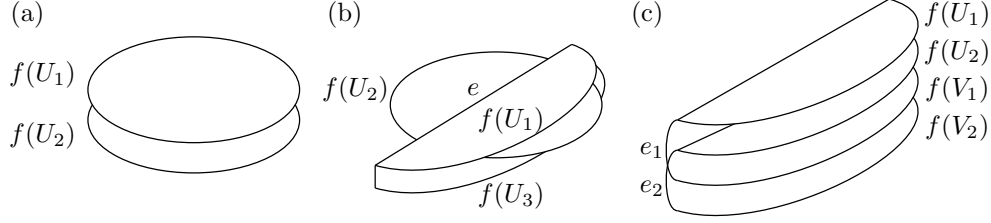


FIGURE 1. (a) The tortilla-tortilla condition being satisfied. (b) The taco-tortilla condition *not* being satisfied. (c) The taco-taco condition *not* being satisfied.

2.1. Background. We follow a model and terminology for planar, two-dimensional flat origami as presented in [CD⁺16] and [DO07].¹ A flat-folded piece of paper may be modeled using two structures: an isometric folding map and a layer ordering. An *isometric folding map* is a continuous, piecewise isometry $f : P \subseteq \mathbb{R}^2 \rightarrow \mathbb{R}^2$ where P is closed. The *crease pattern* of f , denoted X_f , is the set of points on P on which f is non-differentiable, union with the boundary of P . One can prove [Hul20, Rob78] that

- X_f is a plane graph on P whose interior edges, which we call *creases*, are straight line segments,
- every interior vertex of X_f has even degree,
- the faces defined by the embedding of X_f on P are 2-colorable, where one color class is made of regions of P whose orientation are preserved under f and the other color class faces are orientation-reversed under f , and
- around each interior vertex v of X_f the alternating sum of the sector angles between the creases at v , say going in order counterclockwise, equals zero (this is called *Kawasaki's Theorem*).

We will use Kawasaki's Theorem throughout our proofs, and so we formalize what it states for a single vertex in a crease pattern:

Theorem 3 (Kawasaki's Theorem [Hul20, Theorem 5.37]). *A collection of line segments or rays that share a common endpoint $v \in \mathbb{R}^2$ and whose consecutive sector angles are $\alpha_1, \dots, \alpha_2$ will be flat-foldable (meaning they are part of a crease pattern X_f for some isometric folding map f) if and only if $\sum (-1)^k \alpha_k = 0$. Since our crease patterns exist in a flat plane, this is equivalent to*

$$\alpha_1 + \alpha_3 + \dots + \alpha_{2n-1} = \alpha_2 + \alpha_4 + \dots + \alpha_{2n} = \pi.$$

Modeling flat-folded origami also requires the concepts of layer ordering and mountain-valley creases, which require additional structure be added to an isometric folding map. First, we introduce some terminology. A simply connected subset of $U \subset P$ is called *uncreased* under f if f restricted to U is injective. Two simply connected subsets $U_1, U_2 \subset P$ *overlap* under f if $f(U_1) \cap f(U_2) \neq \emptyset$, and we say that U_1 and U_2 *strictly overlap* under f if $f(U_1) = f(U_2)$.

A *global layer ordering* for an isometric folding map f is a function $\lambda_f : \mathcal{C} \subset P \times P \rightarrow \{-1, 1\}$ that records which points of P are above/below which others when folded under f , with $\lambda_f(p, q) = 1$ meaning that p is below q and $\lambda_f(p, q) = -1$ meaning p is above q . Specifically, λ_f is a global layer ordering if the following six properties are satisfied (adopted from [CD⁺16]):

- **Existence:** The domain $\mathcal{C}$ is defined as all $(p, q) \in P \times P$ such that $f(p) = f(q)$. That is, the layer ordering λ_f only exists between points that overlap in the folding.
- **Antisymmetry:** $\lambda_f(p, q) = -\lambda_f(q, p)$ for all $(p, q) \in \mathcal{C}$. That is, if p is above q then q is below p .
- **Transitivity:** If $\lambda_f(p, q) = \lambda_f(q, r)$ then $\lambda_f(p, r) = \lambda_f(p, q)$. That is, if q is above p and r is above q , then r is above p .
- **Tortilla-Tortilla Property (Consistency):** For any two uncreased, simply connected subsets $U_1, U_2 \subset P$ that strictly overlap under f , λ_f has the same value for all $(p, q) \in U_1 \times U_2$. I.e., if two regions in P completely overlap under f , then one must be entirely above the other. This is illustrated in Figure 1(a).

¹The flat-folding of manifolds in general dimension is also possible and follows many of the properties of the flat, 2D case presented here. See [Rob78] and [Hul20, Chapter 10] for more details.

- **Taco-Tortilla Property (Face-Crease Non-crossing):** For any three uncreased, simply connected subsets $U_1, U_2, U_3 \subset P$ such that (a) U_1 and U_3 are separated by an edge e in X_f (i.e., adjacent regions in X_f) and strictly overlap under f and (b) U_2 overlaps the edge e under f , then $\lambda_f(p, q) = -\lambda(q, r)$ for any points $(p, q, r) \in U_1 \times U_2 \times U_3$. I.e., if a region overlaps a nonadjacent internal crease, the region cannot lie between the regions adjacent to the crease in the folding. This is illustrated in Figure 1(b).
- **Taco-Taco Property (Crease-Crease Non-crossing):** If we have uncreased, simply connected adjacent subsets U_1 and V_1 of P separated by a crease e_1 in X_f and U_2 and V_2 separated by a crease e_2 such that the subsets all strictly overlap under f and the creases e_1 and e_2 strictly overlap under f , then for any point $(p, q, r, s) \in U_1 \times V_1 \times U_2 \times V_2$ either $\{\lambda_f(p, r), \lambda_f(p, s), \lambda_f(q, r), \lambda_f(q, s)\}$ are all the same or half are +1 and half are -1. I.e., if two creases overlap in the folding, either the regions of paper adjacent to one crease lie entirely above the regions of paper adjacent to the other crease, or the regions of one nest inside the regions of the other. This is illustrated in Figure 1(c).

global layer ordering ensures that if an actual piece of paper P is to be folded according to an isometric folding map f as determined by its crease pattern X_f , then this can be done without P intersecting itself. This is the generally-accepted definition of what it means for a crease pattern to be *globally flat-foldable* [CD⁺16, DO07, Hul20, Jus97].

An isometric folding map f and global layer ordering λ_f determine a dichotomy for the creases of X_f , called the *mountain-valley (MV) assignment* for (f, λ_f) . Specifically, if a crease e of X_f is bordered by faces U_1 and U_2 and $p \in U_1, q \in U_2$ are close to e with $f(p) = f(q)$, then

- if the orientation of U_1 is preserved under f and $\lambda_f(p, q) = 1$, then e is a *valley* crease, and
- if the orientation of U_1 is preserved under f and $\lambda_f(p, q) = -1$, then e is a *mountain* crease.

Mountain and valley creases correspond to what we see in physically folded paper, where paper bends in either the $\wedge$ (mountain) or $\vee$ (valley) direction. A fundamental result about the mountains and valleys that meet at a flat-folded vertex, which we will often use in our proofs, is *Maekawa's Theorem*:

Theorem 4 (Maekawa's Theorem [Hul20, p. 81]). *If a crease pattern flat-folds around a vertex, the difference between the number of mountain folds and the number of valley folds meeting at that vertex must be 2.*

A generalization of Maekawa and Kawasaki's Theorems that we will need in the proof of Proposition 8 below is the following:

Theorem 5 (Justin's Theorem [Jus97, Hul20, p. 116]). *Let γ be a simple, closed, vertex-avoiding curve on a flat-foldable crease pattern. Let α_i be the signed angles, in order, between the consecutive creases that γ crosses, for $1 \leq i \leq 2n$. Also let M and V be the number of mountain and valley creases, respectively, that γ crosses. Then,*

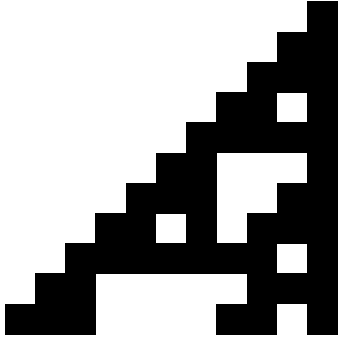
$$(2.1) \quad \alpha_1 + \alpha_3 + \dots + \alpha_{2n-1} \equiv \alpha_2 + \alpha_4 + \dots + \alpha_{2n} \equiv \frac{M - V}{2} \pi \pmod{2\pi}.$$

Computing a global layer ordering, or determining that none exist, for a given isometric folding map is computationally intensive and the main reason why the global flat-foldability problem is NP-hard [BH96].

A useful tool that we will employ (specifically in the proof of Lemma 17 below) is the *superposition net* (or *s-net* for short) that Justin introduced in [Jus97]. The s-net is a superset of the crease pattern X_f of an isometric folding map given by $S_f = f^{-1}(f(X_f))$. That is, S_f is the pre-image of the folded image of the crease pattern. This is helpful because the points of S_f are places where the tortilla-tortilla, taco-tortilla, or taco-taco properties might fail. See [Hul20, Section 6.5] for more details.

2.2. Rule 110 and our conventions. Rule 110 is an elementary (1-dimensional) cellular automaton using the rule table shown in Figure 2. We model a 1 as TRUE and a 0 as FALSE, or black and white pixels, as in Figure 2 where each 1-dimensional state of the cellular automaton is stacked vertically to show the step-by-step evolution of the system. Note that if all inputs are set to 0 the automaton stays constant at 0.

We will simulate Rule 110 in an origami crease pattern by establishing conventions by which the creases can be interpreted as storing and manipulating Boolean variables. In a similar strategy to prior work on origami complexity [CD⁺16, CD⁺20, BH96], we use *directed pleats* (sequences of parallel mountain/valley crease lines) to send TRUE/FALSE signals across the folded paper; we call such directed pleats *wires*. Our



Current pattern	111	110	101	100	011	010	001	000
New state	0	1	1	0	1	1	1	0

FIGURE 2. The table for Rule 110 and ten rows of its evolution from a single TRUE pixel.

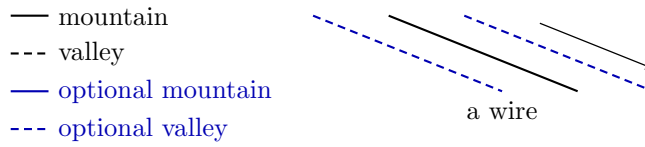


FIGURE 3. guide to the mountain/valley labeling conventions uses in our figures and an example of a wire.

wires are triplets of parallel creases, with a mandatory mountain in the middle and optional valleys to the left and right. We will orient our crease patterns so that the direction of all wires is in the “downward,” decreasing y direction in ². The value of a wire is decided as follows: If the pleat is folded to the right relative to its direction then it is FALSE; if it is folded to the left then it is TRUE. The information in a wire consists of the choice of which valley fold is used.

The labeling conventions we will use in the crease pattern figures in this paper are shown in Figure 3, with solid lines depicting mountain creases and dashed lines being valleys, which is standard in much origami literature. Also, black creases will be mandatory and blue will be optional. An illustration of a wire and its direction is also shown in Figure 3.

In this paper we do not prove that the given crease patterns will fold flat in accordance to the stated operation; that can be verified directly via folding. In our proofs we will simply verify that the given crease patterns will *not* fold in other ways.

2.3. Fundamental results.

Definition 6. In a crease pattern with optional creases, a crease is *active* in a flat-folding if it is used.

Definition 7. In this paper we will be working on an infinite triangular grid of triangles with side-length 1. For any line on the grid, the *next line* in some direction is the closest line in that direction which is parallel to it; if ℓ, ℓ' are two lines and ℓ' is the next line then ℓ and ℓ' are *consecutive*.

wire is three consecutive creases, where the middle crease is a mountain crease, and the two outer creases are optional valley creases.

gadget is a subset of a crease pattern in a region of the plane bounded by a simple closed curve such that the only creases that intersect the boundary are wires.

Note that in order for a wire to have a well-defined Boolean value, we need exactly one of its optional valley creases to be active. However, by themselves any wire could have all three of its creases folded, and thus we want the gadgets that wires enter and exit to force the wires to have one, and only one active crease. The below Proposition will help ensure this.

Proposition 8. Let G be a gadget with angles $\theta_1, \dots, \theta_k$ between consecutive boundary wires (calculated as one transverses the boundary counter-clockwise). Suppose that for all nonempty proper subsets $J \subsetneq \{1, \dots, k\}$

we have

$$\theta_j \not\equiv 0 \pmod{\pi}.$$

If we pick optional creases to make a sub-crease pattern $G' \subset G$ be flat-foldable then each wire in G' contains exactly one active valley fold.

Proof. The angle between consecutive creases in a wire is 0. In addition, each wire contains at least one active crease (the mountain crease). Applying Justin's Theorem to our sub-crease pattern G' with γ the gadget's boundary and $\alpha_1, \dots, \alpha_{j-1}$ the angles in counterclockwise order between the creases in G' that γ crosses, we have $\alpha_1 + \alpha_3 + \dots + \alpha_{j-1} \equiv \alpha_2 + \alpha_4 + \dots + \alpha_{j-2} \equiv m\pi \pmod{2\pi}$ for some constant m . Now, each α_i equals some θ_j or is zero, so our assumption that no proper subset of the θ_j 's adds up to a multiple of π must also be true for the non-zero α_i 's. Therefore, the non-zero α_i 's all have either i being odd or all have i being even. But if both (or neither) of the valley folds in a wire of G were active in G' , then the angles θ_j and θ_{j+1} surrounding the wire would appear among the α_i angles with one having an odd i and one an even i , which is a contradiction. Thus every wire in G' contains exactly one active valley fold.

As a corollary we get:

Corollary 9. *Let G be a gadget with two input wires and two output wires, with consecutive angles between wires adding up to 2π . If we know that the input wires each contain exactly one active valley fold, the output wires must each contain exactly one active valley fold.*

3. LOGIC GATES AND OTHER GADGETS

In this section we show how to construct logical gates (AND, OR, NAND, NOR, NOT) as well as intersection, twist, and eater gadgets via origami with optional creases. These will form the building blocks of our Rule 110 flat origami simulation. Our over-all scheme is to build our crease pattern on a triangle lattice, and so most of our gadgets will possess triangle or hexagonal symmetry. We also include gadgets for some logic gates (AND and NOT) merely for completeness, as they are not used in the final construction.

For the two-input logic gates we assume that we are given two wires at an angle of $2\pi/3$ with information coming "in" to the gate from the positive y -direction. The output is a crease at an angle of $2\pi/3$ with each of the inputs. The NOT gate (Section 3.3) is somewhat strange, as it requires an "auxilliary" pleat which is not affected by the gate. In addition, in Section 3.4, we show that it is possible to "intersect" two wires which meet at an angle of $\pi/3$ without affecting their values. Sections 3.5 and 3.6 contain the twist and eater gadgets.

In the below Propositions, we say that a logic gadget *works* if the values of the input wires force the correct output wire value in accordance to the desired logic gate.

3.1. NOR and NAND.

Proposition 10. *The NOR gate (Figure 4) works.*

Proof. First, suppose that the upper-right input is TRUE. Thus the crease at $\pi/6$ from X is active, and therefore so is the crease at $-5\pi/6$ from X , since only one of the optional valley creases through X may be used by Kawasaki's Theorem. That is, the crease at $-\pi/2$ cannot be active, and thus the output is FALSE.

Before considering the other two cases, a basic observation. Consider the following crease pattern around a point shown in Figure 6. This will not fold flat, since the wedge which is $\pi/6$ wide is too narrow: the wedges on either side of it will collide if we try to flat-fold it. Thus such a configuration is impossible.

Now suppose that both inputs are FALSE. By Kawasaki's Theorem around Y , the fold YY' must be active while none of the others can be (since the fold at $\pi/6$ is active and the fold at $5\pi/6$ is not). By Maekawa's Theorem neither of the other two valley folds through Y' can be active. This means that the folds at $\pi/3$, $2\pi/3$ and $5\pi/6$ through O are not active. From this it follows that none of the folds through O can be active, since otherwise they must all be and then they exactly form the impossible configuration in Figure 6. Thus the folds at 0 , $\pi/3$ and $\pi/2$ through Z are not active, and therefore the fold at $-\pi/2$ through Z cannot be active. Thus the output is TRUE, as desired.

Lastly, suppose that the left-hand input is TRUE and the right-hand input is FALSE. Then the folds at $\pi/6$ and $5\pi/6$ through Y are active, and therefore, for Kawasaki's and Maekawa's Theorems to hold around

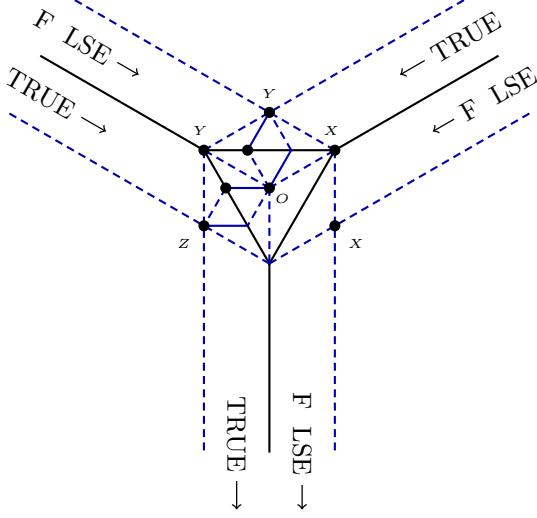


FIGURE 4. NOR gate

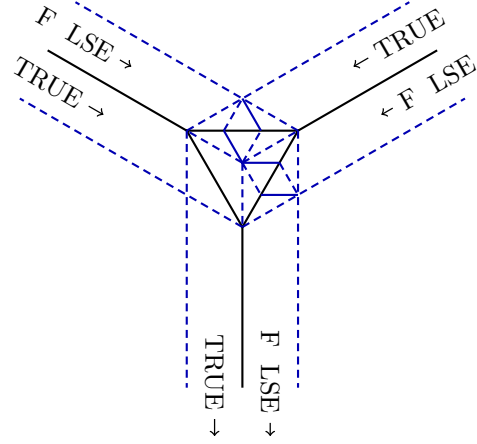


FIGURE 5. NAND gate

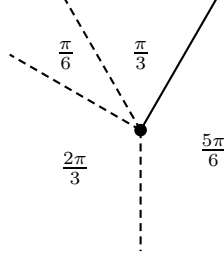


FIGURE 6. An impossible MV assignment in a NOR gate

Y , the folds at $-\pi/3$ and $-2\pi/3$ must be active and YY' and YX' not active. Thus the folds at $\pi/3$ and $2\pi/3$ through O are active. In order for Kawasaki's Theorem to hold around O , we must have the fold at $-\pi/2$ active, and exactly one of the two folds OY' and OX' active. As the configuration in Figure 6 is impossible, the fold OY' cannot be the one that is active, and thus OX' must be the one that is active. By Maekawa's Theorem the fold $Y'Z$ must therefore be active, and thus also the fold at $-\pi/2$ through Z , showing that the output is F LSE, as desired.

Proposition 11. *The NAND gate (Figure 5) works.*

Proof. The NAND gate is a reflection in a vertical line of the NOR gate; since reflection is orientation-reversing in the horizontal direction and orientation-preserving in the vertical direction, by Proposition 10, the output of the NAND gate with inputs A and B is

$$\neg(\neg A \text{ NOR } \neg B) = \neg A \text{ OR } \neg B = A \text{ NAND } B,$$

as desired.

3.2. OR and AND.

Proposition 12. *The OR gate (Figure 7) works.*

Proof. As before, by Proposition 8 exactly one of the output valley folds must be active.

First, suppose that the left-hand input is F LSE. Then the crease at $5\pi/6$ from Y is not active, so the only creases out of Y that can be active are YY' and the crease at $\pi/6$, which are either both active or not. Thus if the right-hand input is TRUE then none of the creases through Y are active. Since none of the creases through Y are active, YY' is not active, and thus neither is $Y'Z$. Thus the crease at $-\pi/2$ through

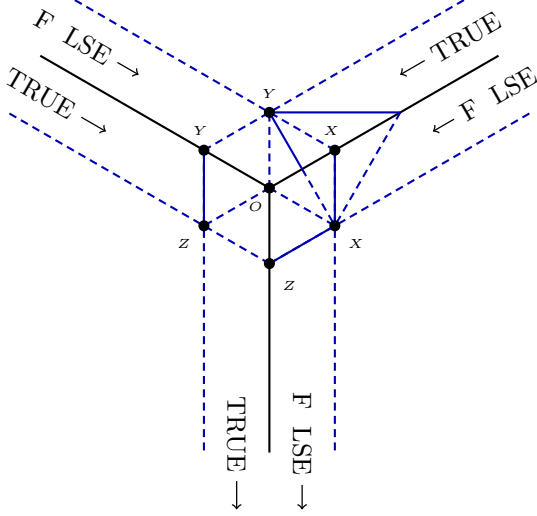


FIGURE 7. OR gate

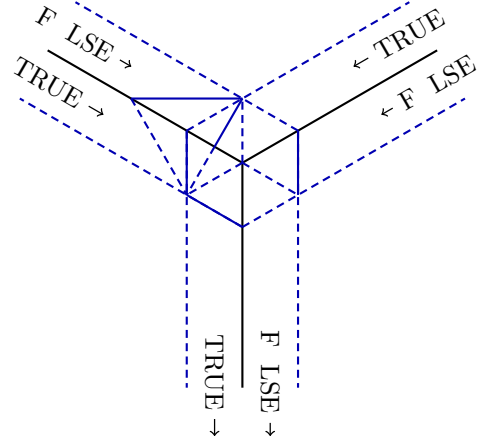


FIGURE 8. ND gate

Z cannot be active, and the output is TRUE. On the other hand, if the right-hand input is F LSE then YY' must be active, and thus therefore so must $Y'Z$. Thus the crease at $-\pi/2$ through Z must be active (since the crease at $5\pi/6$ through Z is also active), and the input must be F LSE.

Now suppose that the left-hand input is TRUE. Then the crease at $5\pi/6$ through Z is not active. Thus ZO and ZZ' cannot be active, and the crease at $-\pi/2$ through Z is active if and only if ZY' is active, which is active if and only if YY' is active. Thus to show that the output is TRUE it suffices to check that YY' is not active. If the right-hand input is TRUE then the crease at $\pi/6$ through Y is not active; by Kawasaki's theorem YY' must not be active in this case, as desired. If the right-hand input is F LSE then the crease at $\pi/6$ through Y is active. If YY' were active then by Kawasaki's theorem YO must be active, and none of the other creases through Y can be. But this violates Maekawa's theorem, since it has 4 valley and no mountain creases meeting at a point. Thus YY' must not be active in this case either, as desired.

Proposition 13. *The ND gate (Figure 8) works.*

Proof. The ND gate is a reflection of the OR gate. By the same logic as in the proof of Proposition 11, since the OR gate works, so does the ND.

3.3. NOT gate. This NOT gate is not used in our origami construction of Rule 110, but we include it for completeness.

Proposition 14. *The NOT gate (Figure 9) works.*

Proof. Suppose the input is TRUE. Then the crease L_4 above the point D is not used, which implies that all of L_2 to the left of point X is used and X is not used (in order to make X flat-foldable). Thus, since the creases L_1 above and L_2 to the right of are both used, we have that both of the short diagonals below are used. This implies that only the lower-left-to-upper-right longer diagonal between and B is used (this diagonal is forced by the short diagonals below; the other diagonal between and B cannot also be used because we cannot have a degree-4 vertex made of only valley creases). This implies that the two short diagonals above point B are not used, which means the crease L_3 to the right of point Y is used and L_1 below B is not used. Therefore the output is F LSE. By the left-right mirror symmetry of this crease pattern, if the input is F LSE the output must be TRUE.

3.4. Intersector. Intersector gadgets will be placed wherever two wires need to cross each other on our sheet of paper. They will ensure that the Boolean signals of the wires will be preserved through the intersection.

Proposition 15. *The intersectors (Figures 10 and 11) work.*

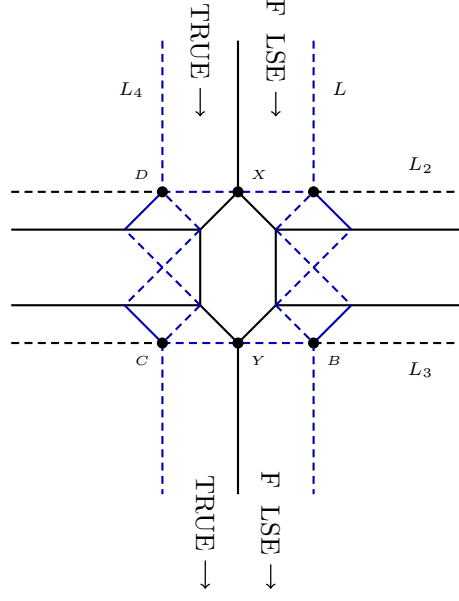


FIGURE 9. NOT gate

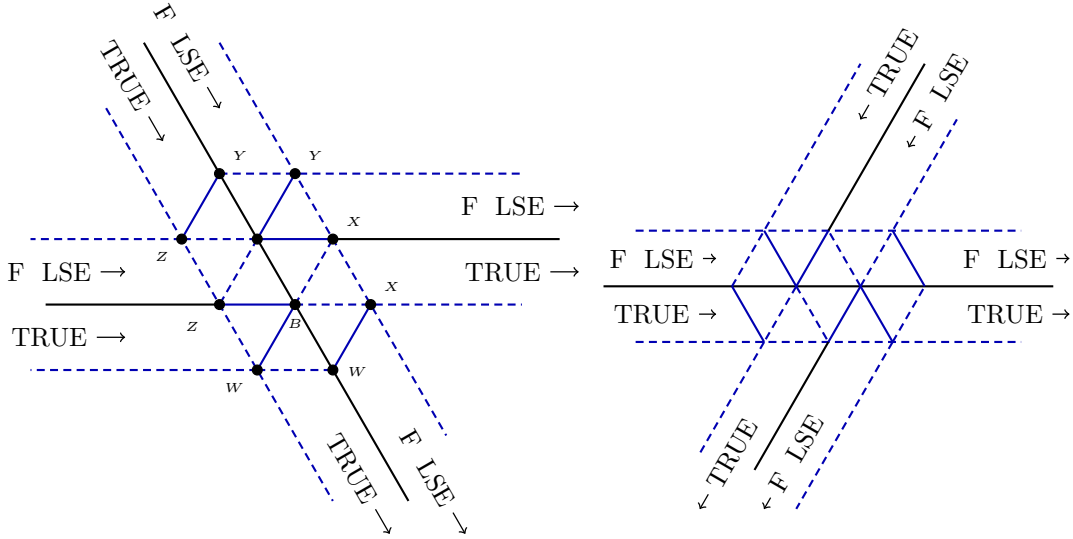


FIGURE 10. $\pi/3$ Intersector

FIGURE 11. $2\pi/3$ intersector

Proof. It suffices to check the claim for the $\pi/3$ intersector; the analysis for the $2\pi/3$ intersector is completely analogous.

By Corollary 9, exactly one of the valley folds in each of the output wires will be active.

First, suppose that both inputs are F LSE. Then at point Z one of the angles between active creases is π , and thus both creases ZY' and Z are not active, and ZZ' is active. Since ZY' is not active, neither is $Y'Y$, and since the crease at $2\pi/3$ from Y is not active either, none of the creases through Y are active. Since YX' is not active, X' must be active; thus crease Z' must be active. Since Z' and ZZ' are active, crease $Z'B$ must not be active, and crease $Z'W$ must be active. Since $Z'W$ and the crease at π through W are active, WB must be active and the crease at $-\pi/3$ through W must be active. Thus both outputs are F LSE, as desired.

Now suppose that the left-hand input is TRUE and the top input is F LSE. Then creases ZY' and ZZ' must also be active, as well as crease $Y'Y$. Since $Y'Y$ is active but the crease at $2\pi/3$ from Y is not, the crease at 0 from Y must be, and the right-hand output is TRUE. Since ZZ' is active, $Z'B$ must not be, but all other valley folds through Z' will be. Since $Z'W$ is active but the crease at π from W is not, the crease at $-\pi/3$ at W must be; thus the bottom output is F LSE and the gadget folds as claimed.

Now suppose that the left-hand input is F LSE and the top input is TRUE. Then neither the crease at $2\pi/3$ nor the crease at π through Z are active, and thus none of the creases through Z are active. Since Z is not active, crease Y cannot be active either; thus crease YX' is active, but the crease at 0 through Y is not active; therefore, the right-hand output is F LSE. Since YX' is active, X' cannot be active, and thus $X'B$ and $X'X$ must be active. But Z' is not active, and thus $Z'B$ must be active, while $Z'W$ must not be. Considering point W we see that since the crease at π through W must be active, crease WW' must also be active, and therefore so is $W'X$. Since $X'X$ and $W'X$ are active, the crease at 0 through X must also be active. Now there are two possible configurations that seem to work: just the crease at $-\pi/6$ through X active (giving the desired output), or else creases XB , BW , and the crease at $-\pi/6$ through W active—giving the incorrect output. However, the latter of these is exactly the configuration considered in Lemma 17, which shows that it cannot flat-fold. Thus the only possible configuration that flat-folds is the correct one.

Lastly, suppose that both inputs are TRUE. Since the crease at π through Z is active but the one at $2\pi/3$ is not, crease Z must be active, and therefore so must Y . Since crease Y is active, three of the valley folds through Y must be active, and one of them must be the crease at 0 through Y ; thus the right-hand output is TRUE. We claim that YY' cannot be active. Indeed, suppose that it is. Then $Y'Z$ must also be, and therefore so is ZZ' . Then Z' and $Z'W$ must be active, but $Z'B$ not. Since $Z'W$ is active but the crease at π through W is not, the crease at $-\pi/3$ must be, while WB and WW' (and thus $W'X$) are not. Since $Z'B$ and WB are not active, BX' and BX must not be, either. Since four of the creases at Y are active, crease YX' must not be; thus X' must be active, and $X'X$ is not. Thus none of the creases at X can be active. This is exactly the problematic configuration discussed in Lemma 17, which cannot flat-fold.

Thus YY' is not active. Then neither are $Y'Z$ or ZZ' . Since ZZ' is not active, $Z'B$ must be active and Z' and $Z'W$ not active. Since $Z'B$ is active, BX' must be active and X' must not be. Thus both YX' and $X'X$ are active. Since the crease at 0 through X is not active, the crease at $-\pi/3$ must be, and the bottom output is TRUE, as desired.

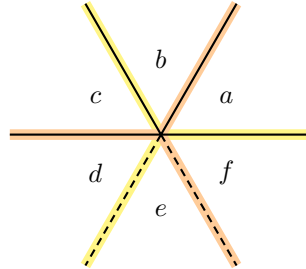


FIGURE 12. Hexagonal folder

Write $x < y$ to mean that layer y is above layer x .

Lemma 16. *The only flat-foldings of Figure 12 in which region e is upward-facing have layer orderings (from top to bottom) $c > d > a > f > e > b$ and $a > f > c > d > e > b$.*

Proof. Since the crease pattern is symmetric with respect to a vertical reflection, we can assume without loss of generality that $c > a$. Now, since e is upward-facing, so is c and a , and thus the MV assignment in Figure 12 implies that any flat-folding of this vertex must have $c > d > e$, $a > f > e$, and $c, a > b$; since we assumed $c > a$, this implies that $c > a > b$. In order to avoid self-intersections, we note that after folding all of the orange creases will be lined up, and the yellow creases will be lined up. To figure out possible orderings, we build up the layer orderings in stages.

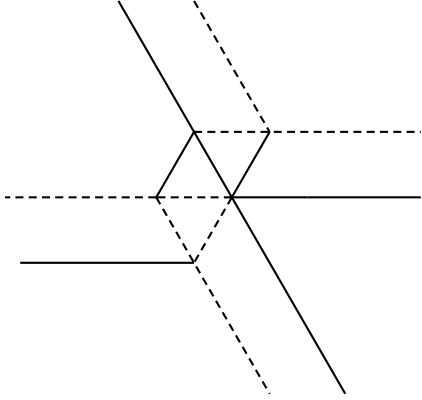


FIGURE 13. Problematic Intersector configuration

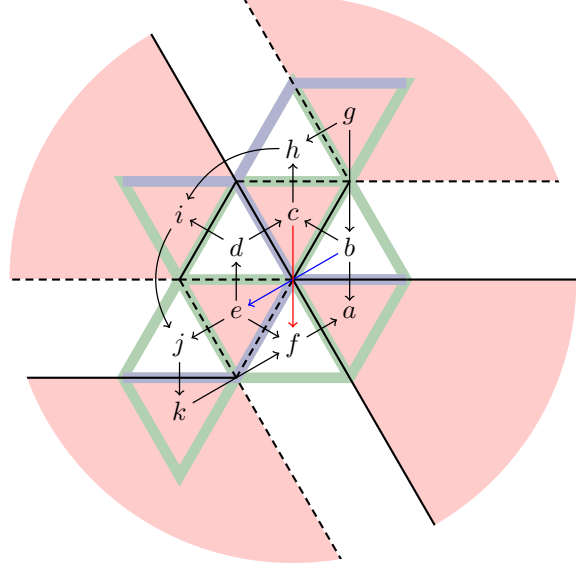


FIGURE 14. Problematic Intersector configuration with annotations

The first stage is that $c > d > e$ coming from the mountain-valley pattern. To build the second stage, consider the possibilities for how $a > f > e$ can interleave with this ordering. Suborderings of the form $a > d > f > e$ and $c > f > d > e$ are not allowed, by the taco-taco property (see Section 2.1) around the yellow and orange edges, respectively. Thus f cannot go between c and d ; since $a > f$, we must therefore have $c > f$, and thus the only possible ordering is $c > d > a > f > e$. For the third stage, consider where b can be inserted in this ordering. Since $a > b$ the only possibilities are

$$c > d > a > b > f > e \quad c > d > a > f > b > e \quad c > d > a > f > e > b.$$

The first of these is forbidden by the taco-taco property around the yellow edges; the second is forbidden by the taco-taco property around the orange edges. Thus the only possibility is the last ordering (which is possible by first folding the top half behind the bottom half, then folding the right half down, and then the left).

Lemma 17. *The problematic configuration (Figure 13) will not fold flat.*

Proof. This proof will refer to the annotated version, Figure 14. The colored regions in the diagram are upward-facing; the others are downward-facing. The graph on the nodes shows ordering relations enforced on the regions in the graph. (The green and blue lines show a part of the s-net (defined in Section 2.1) consisting of the parts of the s-net which overlap region c after folding; a flat-folding must give a well-defined ordering on these regions.) The directions in the graph points from a layer that must be lower to one that must be higher. cycle in the graph thus exhibits a contradiction.

The black edges in the graph are drawn from local mountain-valley conditions: an upward-facing layer must be above a neighboring layer if they differ by a mountain fold, and below if they differ by a valley fold. The blue edge in the graph follows from Lemma 16 applied to regions a, b, c, d, e, f . Note that there is a path from c to a ; thus if we wish to avoid cycles, the layer ordering must have $c < a$. Thus by Lemma 16 the ordering on these layers must be $a > f > c > d > e > b$, giving the red arrow.

analyzing the black graph with this data, we see that we must have

$$g < b < e < d < c < h < i < j < k < f < a.$$

However, at the top edge of i there is a taco-tortilla condition which is violated. Layer i is between layers a and b , but does not contain a crease line at the blue line. The blue line is lined up in the folding map with the crease between a and b , giving the contradiction. Thus the crease pattern cannot flat-fold.

3.5. Twists. *twist fold* is a crease pattern where n pairs of parallel mountain and valley creases (pleats) meet at an n -gon such that folding the pleats flat results in the n -gon rotating when folded flat. In addition, standard twist folds require the pleats to fold in the same rotational direction, either all clockwise (with the pleats folding mountain-then-valley in the clockwise direction) or all counterclockwise (valley-then-mountain in the clockwise direction). Twist folds are ubiquitous in origami tessellations; see, e.g., [Gje08].

Triangle and hexagon twists were pioneered by Fujimoto in the 1970s [FN82]. Such twists with optional valley creases so as to allow the triangles/hexagons to rotate in either the clockwise or counterclockwise direction are shown in Figures 15 and 16.

We will use these triangle and hexagon twists as gadgets to propagate wire signals in various directions, while also negating them. Their forced rotational nature proves the following Proposition; they are simple enough to analyze that we omit the details of the proofs.

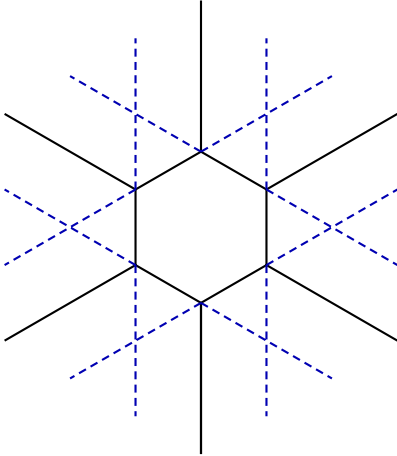


FIGURE 15. Hexagonal twist

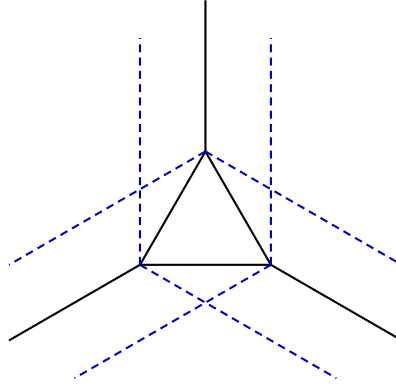


FIGURE 16. Triangle twist

Proposition 18. *The pure hexagonal and triangle twists (Figures 15 and 16) must flat-fold in rotationally-symmetric ways. In particular, given any designated wire in the top half of the gadget as the input, both twists negate and duplicate the input value in the other, output wires.*

3.6. The eater. Sometimes twist folds will produce extraneous wires, or *noise* wires that are not needed for our construction of Rule 110. In order to eliminate these we have *eater gadgets* that accept any combination of Boolean wire values. In particular, if it can be arranged that all of the noise wires in adjacent cells either cancel one another out (by colliding in an eater) or match up, then the cells can be tessellated. modified triangle twist, shown in Figure 17 does the job nicely.

Proposition 19. *The values of the three wires entering the eater (Figure 17) are independent. In other words, the eater will flat-fold regardless of the values of the three wires.*

Proof. Since the eater crease pattern is rotationally-symmetric as well as reflectively-symmetric about each mountain input axis, all one needs to do to show that the eater will fold flat for any set of inputs is to check when the inputs are all TRUE or have two TRUE and one FALSE inputs. When all are TRUE the eater turns into a triangle twist and thus can fold flat. The TRUE, TRUE, FALSE case requires using the short optional mountain crease (and its collinear optional valley) that is between the adjacent optional valleys between one of the TRUE and the FALSE inputs. It can be readily checked that this, too, is flat-foldable.

4. FOLDING RULE 110

4.1. The Main Theorem. Figure 18 shows the schematic of a crease pattern that logically simulates Rule 110. This crease pattern is overlaid on the triangle lattice for clarity; the underlying triangle lattice is not part of the crease pattern. The vertical wires labeled A, B, and C along the top of the Figure are the inputs and the center-most vertical wire at the bottom, labeled OUT, is the output. The wires and gadgets drawn

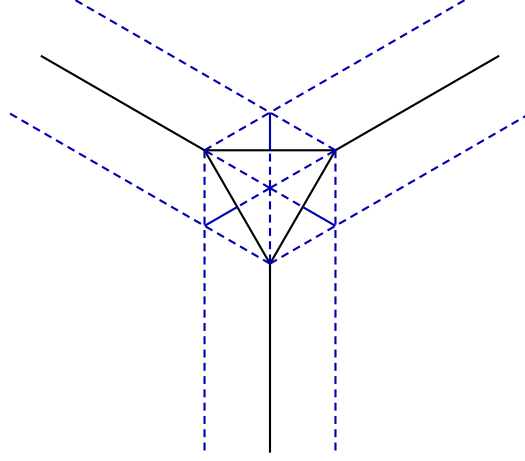


FIGURE 17. Eater

in color in the Figure control the logical workings of this crease pattern. The wires and gadgets drawn in grey absorb and direct the noise wires generated by the crease pattern. Also note that the numerous gadgets detailed in the previous Section are simply labeled in Figure 18 by their names, like ND, OR, E (for the eater gadget), and so on rather than drawing all the individual creases. The pale yellow hexagons and triangles are hexagon and triangle twists, respectively.

Theorem 20. *If the crease pattern in Figure 18 is given mandatory creases for the input wires A , B , and C , then optional creases can be chosen from the rest to make the crease pattern fold flat, and the result will force truth value of the output wire to follow Rule 110 from the inputs A , B , and C .*

Proof. In the crease pattern, the splitter and crossover gadgets lead the three original inputs, or their negations into a NOR and a pair of N-AND clauses and then pass them into triangle twists to produce the following three signals (labeled in Figure 18):

$$X = A \vee \neg B \quad Y = \neg B \wedge C \quad Z = B \wedge \neg C.$$

The outputs of these are then led into a N-AND and an OR clause to produce the two signals

$$P = \text{TRUE} \wedge X, Q = \neg(Y \vee Z).$$

Finally, the values of P and Q are led to a N-AND gadget, so that the final output signal is

$$\neg(P \wedge Q).$$

In other words, the output of this crease pattern is:

$$\neg((A \vee \neg B) \wedge \neg((\neg B \wedge C) \vee (B \wedge \neg C))).$$

This simplifies to

$$(4.1) \quad (\neg A \wedge B) \vee ((\neg B \wedge C) \vee (B \wedge \neg C)).$$

This is exactly what Rule 110 performs. That is, the output is TRUE if $B \neq C$ (which makes the second clause in (4.1) TRUE), and if $B = C$ then this second clause will be FALSE and the output will be the value of $(\neg A \wedge B)$. That's Rule 110.

We remark that all of the gadgets used in the construction of the flat-folding simulation of Rule 110 generate a unique MV assignment (and thus a unique set of output wire values) for a given set of input wire values. Thus the same is true for our Rule 110 crease pattern (Figure 18) *except* for the places where two eater gadgets share a wire. Such a shared wire between eater gadgets could be either TRUE or FALSE and not affect the logical constraints of the rest of the crease pattern. Nonetheless, we have demonstrated that *some* flat-folding of the crease pattern will exist to perform Rule 110 computations, which is all that is required to simulate Rule 110.

4.2. On finiteness, flat-foldable origami tessellations, and Turing machines. The goal of this paper is to prove that flat-folding is Turing-complete, but what does it mean to make this statement? A Turing machine is a finitely-defined object (i.e. a machine with a finite number of states) working with infinite storage space (the tape) on which is recorded a finite starting input. In order to translate this into origami, we form a crease pattern made of finite-state cells that are tessellated onto the infinite plane.

Consider the following variation of the global flat-foldability problem: We are given an infinite tessellation of a finite straight-line planar graph G drawn on our paper $P = \mathbb{R}^2$; the tessellation must have a finite fundamental region, which is the cell in question. All the edges of G are labeled as either mandatory mountains, mandatory valleys, optional mountains, or optional valleys. G is equipped with a subgraph G_{init} , which contains all of the mandatory creases and a subset of the optional creases, and which also forms a tessellation (i.e. the chosen subgraph is the same in every cell), such that G_{init} is the crease pattern X_f of some isometric folding map $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ with a global layer ordering λ_f whose MV assignment corresponds to the mountain and valley labeling of G_{init} . This is the “ground state” setup of our flat-folding Turing machine: the blank tape.

To specify a more general problem, we take a graph G' which contains G and which differs from G by only (a) the addition of finitely many creases, or (b) the modification of an optional crease into a mandatory crease. The question becomes: is there a subset H of G' , which contains all of the mandatory creases and a subset of the optional creases, such that H is the crease pattern X_f of some isometric folding map $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ with a global layer ordering λ_f whose MV assignment corresponds to the mountain and valley labeling of H .

The main result of this paper is that this problem is Turing-complete. Theorem 20 shows that Rule 110, a finite cellular automaton which is known to be Turing-complete [Coo04], and in fact P-complete [?, Theorem 1], can be modeled using a tessellating crease pattern satisfying the above conditions. The above-mentioned crease pattern H is the output of the Turing machine from a given input.

The connections between Turing machine computations, Rule 110, and in our flat-foldable crease pattern might seem mysterious to the reader, especially the requirement that such computations be performed with a finite amount of material. Thus, we provide a few details on this in the remainder of this Section.

Definition 21. Consider an elementary cellular automaton with input values encoded as a row of cells, colored black for 1 and white for 0. The computation of the automaton is recorded as a grid with this input row as the top, and each consecutive row determined by the computational rule of the automaton. This gives a coloring of the plane. A *spaceship* is a self-replicating tile: a finite sequence of cells that, if repeated infinitely, will produce the original pattern back in a finite number of computations.

Computation on Rule 110 is done by observing perturbations in a sequence of standard spaceships, which are themselves a tessellation 14 cells wide and 7 cells tall. (See [Coo04, ?] for more details.) Take a 14×7 repetition of the basic set of cells as the basis of the origami tessellation; since the spaceships tessellate infinitely, this will fold flat. To compute with Rule 110 a finite number of perturbations are made to the spaceship pattern; this is encoded in the crease pattern by setting the direction of input wires to be TRUE or FALSE by modifying the appropriate optional crease to be a mandatory crease, as desired, and altering the NAND gate above each input wire to be an eater (by adding a finite number of optional creases; the extra creases present in the NAND will not affect the flat-foldability of the gadget as they are all optional). This gives the input value to the computation below it. The part of the pattern below the given inputs will fold flat with finitely many modifications to the original spaceship pattern if and only if the original input reverts to the standard spaceships after finitely many steps. We conclude that since we need a constant amount of our paper to replicate a Rule 110 spaceship, and by Theorem 20, we have that our generalized flat-foldable origami problem is Turing-complete, and in fact P-complete.

5. CONCLUSION

We have shown that folding origami crease patterns with optional creases into a flat state can emulate the behavior of the one-dimensional cellular automaton Rule 110, and can therefore perform the tasks of a universal Turing machine. Actually folding a piece of paper to simulate, say, multiple rows of an instance of Rule 110 using the crease pattern presented here would be a gargantuan task, even for expert origami

artists. Using these methods to perform the computations of a Turing machine using flat origami would be even more arduous, so this is by no means meant to be a practical way to perform computation via origami.

By way of comparison, we note the existence of prior work from the physics and engineering community on using origami for actual computation, e.g. [SPM17, TGBV18, YLM21]. These studies build logic gates using *rigid origami*, where a stiff material is folded in a continuous motion so that the creases act like hinges and the regions of material between the creases remain planar, or rigid, during the folding process. Determining whether a crease pattern can be rigidly folded in this way has also been proven to be NP-hard [CD⁺20]. While it is likely that rigid origami is also Turing complete as a computational device, to our knowledge no one has proven this. The crease patterns and gadgets in the present work are not rigidly foldable and therefore could not be used as-is in such a proof. Rather, computation performed by flat origami should be viewed discretely, where only the fully flat-folded state provides the desired computational information.

The logic gadgets presented in this paper may be used to simulate other one-dimensional cellular automata. For example, a crease pattern to produce a Sierpinski triangle modulo 2 would be given by iteration of the cell in Figure 19; to make this tessellate it is necessary to reflect consecutive units in each row, and the cells shift half a cell-width in each row. The basic cell, as above, is in a dark green box. To see the Sierpinski effect, one could color the “true” side of the input/output wires blue and the “false” red.

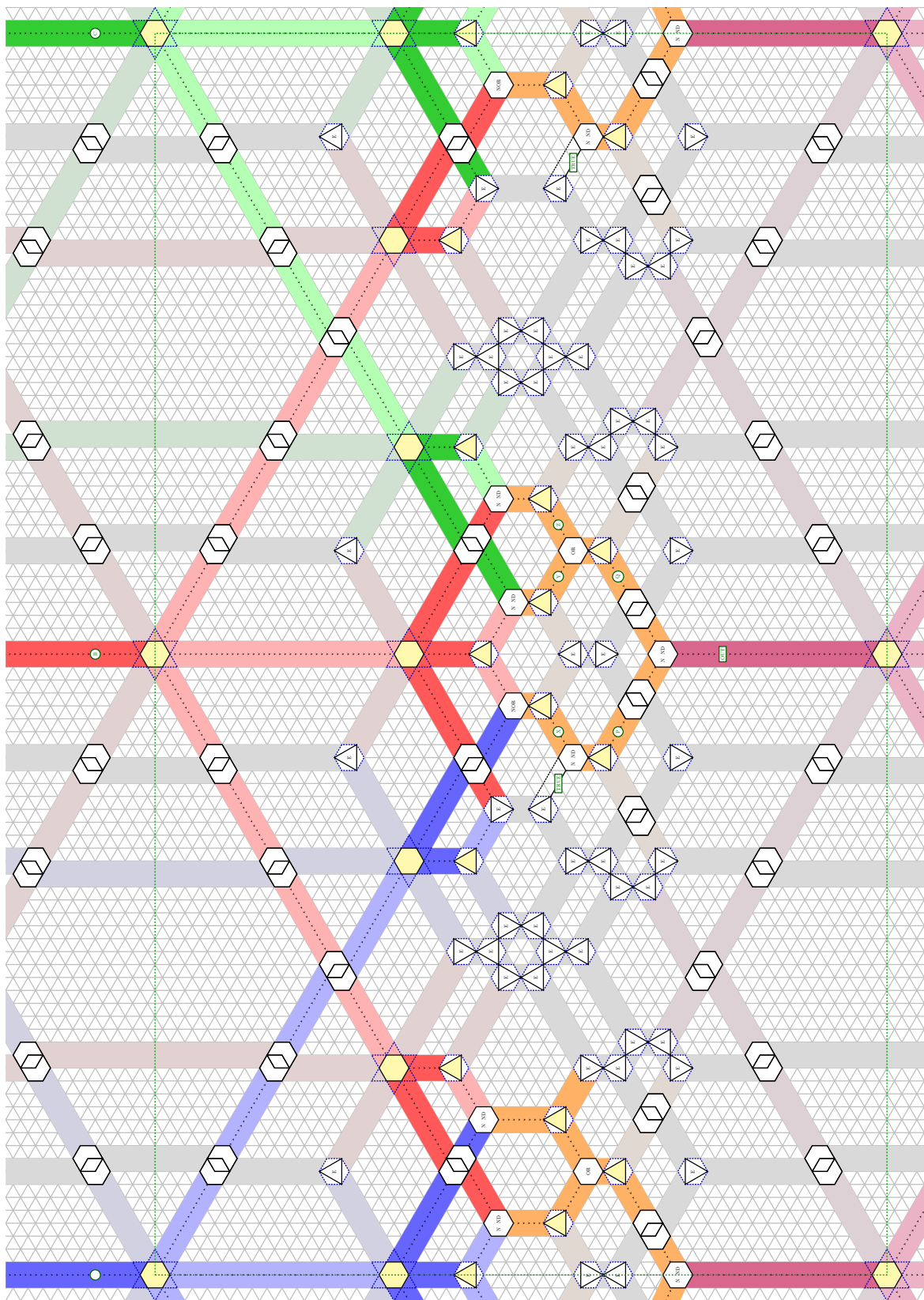


FIGURE 18. Origami crease pattern with optional creases that simulates a Rule 110 cell.

- [SPM17] Menachem Stern, Matthew B. Pinson, and Arvind Murugan. The complexity of folding self-folding origami. *Phys. Rev. X*, 7:041070, Dec 2017.
- [TGBV18] Benjamin Trembl, Andrew Gillman, Philip Buskohl, and Richard Vaia. Origami mechanologic. *Proceedings of the National Academy of Sciences*, 115(27):6916–6921, 2018.
- [YLM21] Wenzhong Yan, Chang Liu, and Ankur Mehta. Origami logic gates for printable robots. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6084–6089, 2021.

FRANKLIN & MARSHALL COLLEGE

Email address: `thomas.hull@falm.edu`

CORNELL UNIVERSITY

Email address: `zakh@math.cornell.edu`